Comprehensive Optimization Guide

This guide walks you through the 5 essential steps to optimize the Hotel Management API for performance, scalability, and security. Follow each step, copy the code provided into the specified files, and run your tests afterward to verify the improvements!

Step 1: Add Database Indexes

What will improve:
Query speed! Currently, if someone searches for hotels in a specific city, Postgres performs a "Sequential Scan"—meaning it reads every single row in the `hotels` table to find matches. If you have 100,000 hotels, it reads 100,000 rows. By adding indexes, the database lookup time will drop from milliseconds (or seconds) down to microseconds.

Real-World Scenario:
Imagine you walk into a massive library and ask the librarian for all books about "Hotels in Paris." Without an index, the librarian has to manually pull *every single book* off the shelf, check if it's about Paris, and then hand you the pile. This takes weeks. With an index, the librarian simply looks in the library's catalog, finds "Paris," and instantly knows exactly which shelves to pull the books from.

Target File: `src/db/schema.ts`

Instructions:

1. Import `index` from the Drizzle ORM package at the top of the file.
2. Add the index arrays to the `hotels`, `rooms`, and `bookings` tables.

```
import {
  pgTable, text, timestamp, pgEnum, uuid, unique,
  varchar, decimal, jsonb, integer, date, check,
  index // <--- 1. Add this import
} from "drizzle-orm/pg-core";

// 2. Update your tables to include the index array as the second argument:

export const hotels = pgTable("hotels", {
  // ... your existing columns ...
}, (t) => [
  index("idx_hotels_city").on(t.city),
  index("idx_hotels_country").on(t.country),
  index("idx_hotels_rating").on(t.rating)
]);

export const rooms = pgTable("rooms", {
  // ... your existing columns ...
}, (t) => [
  unique("hotel_room_number_unique").on(t.hotelId, t.roomNumber),
  index("idx_rooms_hotel_id").on(t.hotelId)
]);

export const bookings = pgTable("bookings", {
  // ... your existing columns ...
}, (t) => [
  check("check_dates", sql`${t.checkOutDate} > ${t.checkInDate}`),
  index("idx_bookings_user_id").on(t.userId),
  index("idx_bookings_hotel_id").on(t.hotelId),
  index("idx_bookings_room_id").on(t.roomId)
]);
```

Step 2: Fix the Express App Security & Size

What will improve:

- **Bandwidth Usage:** `compression` will reduce the size of your JSON responses by up to 80%.
- **Security:** `helmet` prevents common web vulnerabilities (like Cross-Site Scripting), and `express-rate-limit` stops hackers from brute-forcing passwords or scraping your entire database.

Real-World Scenario:

- **Compression:** Imagine shipping 100 fluffy pillows in a huge moving truck. It's slow and expensive. Compression is like vacuum-sealing those pillows into a tiny box before shipping them. When the client's browser receives the box, it "unseals" it instantly.
- **Rate-Limiting:** Imagine a malicious competitor writing a bot that tries 10,000 passwords a second on your login page to steal customer accounts. A rate-limiter acts like a bouncer at the door: "You've tried 100 times in the last 15 minutes. Come back later." It blocks the bot before it even reaches your database.

Target File: `src/app.ts`

Instructions:

1. Open your terminal and install the required packages:

   ```
   bun add helmet compression express-rate-limit
   bun add -D @types/compression
   ```

2. Apply these middleware layers to your Express app.

```
import express from "express";
import type { Request, Response, NextFunction } from "express";
import helmet from "helmet"; // <--- Add this
import compression from "compression"; // <--- Add this
import rateLimit from "express-rate-limit"; // <--- Add this
import { authRoutes, hotelRoutes, bookingRoutes, reviewRoutes } from "@/routes/exportAllRoutes";

const app = express();

// 1. Security Headers (hides tech stack, stops XSS)
app.use(helmet());

// 2. Compress JSON responses (saves massive bandwidth)
app.use(compression());

// 3. Rate limiting (max 100 requests per 15 minutes per IP)
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 100,
  message: { success: false, data: null, error: "TOO_MANY_REQUESTS" }
});
app.use(limiter);

// 4. Parse JSON
app.use(express.json());

// ... rest of your routes ...
```

Step 3: Add Pagination to Hotel Searches

What will improve:
Memory consumption and network latency. Returning thousands of hotels at once requires Node.js to load them all into RAM, convert them to a massive JSON string, and send them over the wire. This will eventually cause an "Out of Memory" server crash.

Real-World Scenario:
Imagine an e-commerce store like Amazon. If you search for "Shoes", Amazon doesn't send you a single webpage with 5 million pairs of shoes loaded all at once; your browser would crash trying to render it. Instead, they send you Page 1 with 20 pairs. If you want to see more, you click "Next Page." Pagination enforces this at the database level.

Target File: `src/controllers/hotelController.ts`

Instructions:

1. Extract `page` and `limit` from the query parameters.
2. Apply `.limit()` and `.offset()` to your Drizzle query.

```
export const searchHotels = asyncHandler(async (req: Request, res: Response) => {
  const { city, country, minPrice, maxPrice, minRating } = req.query;

  // 1. Setup Pagination variables (Default: page 1, limit 20)
  const page = Number(req.query.page) || 1;
  const limitNum = Number(req.query.limit) || 20;
  const offsetNum = (page - 1) * limitNum;

  // ... (keep your cityStr, countryStr setup) ...

  let query = db
    .select({
       // ... (keep your existing select columns) ...
    })
    .from(hotels)
    .innerJoin(rooms, eq(hotels.id, rooms.hotelId))
    .where(/* existing where clause */)
    .groupBy(hotels.id)
    .limit(limitNum)   // <--- 2. Add limit!
    .offset(offsetNum) // <--- 2. Add offset!
    .$dynamic();

  // ... (keep the rest of your code) ...
});
```

(Pro-tip: You should also apply `.limit()` and `.offset()` to your `getUserBookings` function in `bookingController.ts`!)

Step 4: Fix the Double Booking Race Condition

What will improve:
Data integrity and trust. If two users try to book the exact same room for the exact same dates at the exact same millisecond, standard database transactions might allow both to succeed, resulting in an angry double-booked customer. This fix ensures 100% atomic bookings.

Real-World Scenario:
Imagine you and a stranger are both trying to buy the very last ticket to a Taylor Swift concert online at the exact same second.

- **Without row-level locking:** The database tells you both, "Yes, there is 1 ticket left." You both click 'Buy' at the same time. The database accepts both payments, creating a massive issue.
- **With row-level locking (`.for("update")`):** As soon as you ask, "Is there a ticket left?", the database places a "Hold" sign on the ticket. When the stranger asks the

same question a millisecond later, the database tells them, "Wait in line, someone is currently looking at this ticket." Once you buy it, the database updates to 0, and tells the stranger "Sorry, sold out."

Target File: `src/controllers/bookingController.ts`

Instructions:

1. Inside `createBooking`, locate the query where you fetch the room at the start of the transaction.
2. Add `.for("update")` to the end of that query.

```
// Replace your current room fetch with this:
const roomWithHotel = isValidUUID(roomId) ? await tx
  .select({
    room: rooms,
    hotel: hotels,
  })
  .from(rooms)
  .innerJoin(hotels, eq(rooms.hotelId, hotels.id))
  .where(eq(rooms.id, roomId))
  .for("update") // <--- THIS IS THE MAGIC LOCK!
  : [];

// Why? No other transaction can read this specific room row until the current
// transaction finishes, guaranteeing 100% atomic capacity checks!
```

Step 5: Implement In-Memory Caching

What will improve:
API response times will drop from ~50ms down to ~1ms for highly trafficked routes. Your database CPU usage will also drop drastically, as the API intercepts the request and responds before the database is ever queried.

Real-World Scenario:
Imagine running a busy pizza restaurant. A customer calls and asks, "What are your toppings?" The cashier runs back to the kitchen, opens the fridge, reads all 15 toppings, runs back to the phone, and reads them off. (This is querying the database).
If 100 customers call asking the same question, the cashier gets exhausted running to the fridge.
Caching is like taking a sticky note, writing down the 15 toppings, and sticking it next to the phone. For the next 5 minutes, whenever someone asks, the cashier just reads the sticky note instantly without ever walking to the kitchen.

Instructions:

1. Install the caching library in your terminal:

   ```
   bun add node-cache
   bun add -D @types/node-cache
   ```

2. Wrap your `getHotelDetails` logic with the cache getter and setter.

Target File: `src/controllers/hotelController.ts`

```
import NodeCache from "node-cache";

// 1. Initialize the cache to hold items for 300 seconds (5 minutes)
const hotelCache = new NodeCache({ stdTTL: 300 });

export const getHotelDetails = asyncHandler(async (req: Request, res: Response) => {
  const hotelId = req.params.hotelId as string;

  if (!isValidUUID(hotelId)) {
    return res.status(404).json({ success: false, data: null, error: "HOTEL_NOT_FOUND" });
  }

  // 2. Check if the hotel data already exists in the cache (Checking the sticky note)
  const cachedHotel = hotelCache.get(hotelId);
  if (cachedHotel) {
    console.log(`Cache HIT for hotel: ${hotelId}`);
    return res.status(200).json({
      success: true,
      data: cachedHotel,
      error: null,
    });
  }

  console.log(`Cache MISS for hotel: ${hotelId} - Fetching from DB...`);

  // ... (Keep your existing database queries here!) ...
  const [hotel] = await db.select().from(hotels).where(eq(hotels.id, hotelId));
  if (!hotel) {
     return res.status(404).json({ success: false, data: null, error: "HOTEL_NOT_FOUND" });
  }
  const hotelRooms = await db.select().from(rooms).where(eq(rooms.hotelId, hotelId));

  // 3. Build the final data object
  const hotelData = {
    id: hotel.id,
    // ... all your existing mappings ...
  };

  // 4. Save the result in the cache so the next user gets it instantly! (Writing the sticky note)
  hotelCache.set(hotelId, hotelData);

  return res.status(200).json({
    success: true,
    data: hotelData,
    error: null,
  });
});
```

Final Validation

Once you have applied these 5 steps to your code, run your test suite one last time to confirm that the business logic remains fully intact:

```
bun test
```